

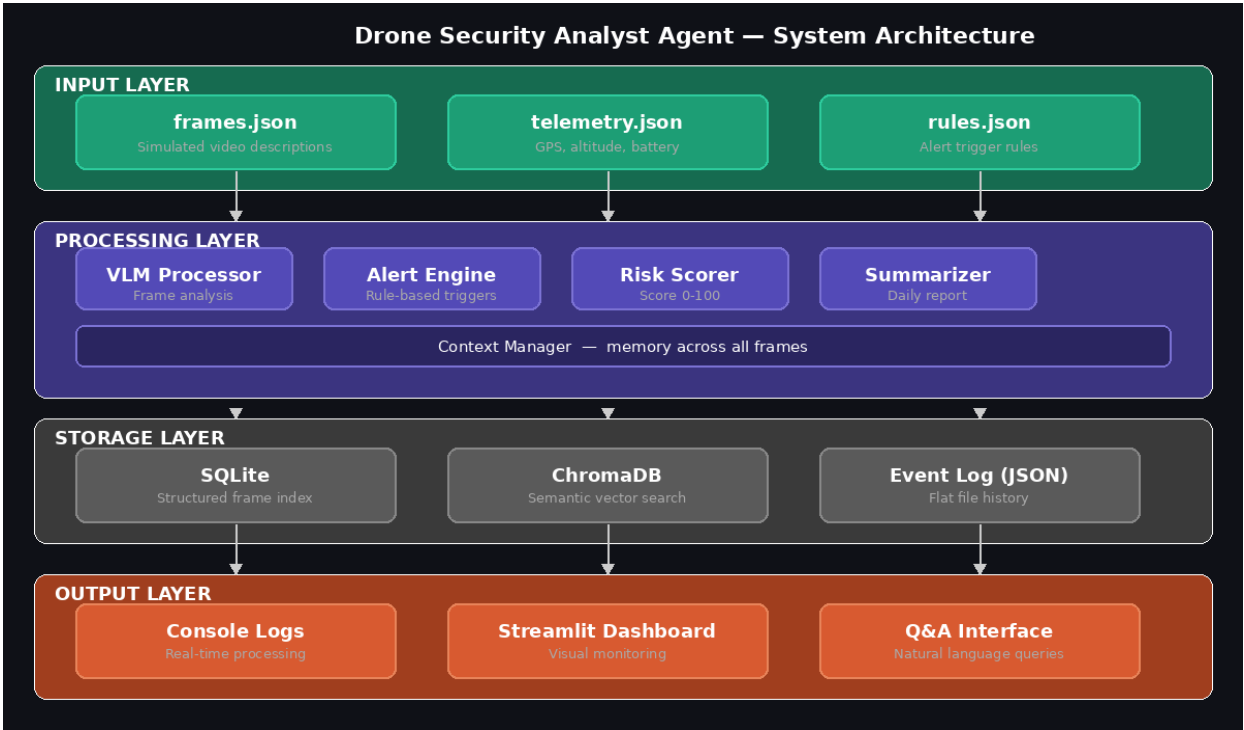
Drone Security Analyst Agent

System Design Document

AI Engineer Assignment — FlytBase

1. System Architecture

The system is organized into four distinct layers — Input, Processing, Storage, and Output. Each layer has a clearly defined responsibility and communicates with the next layer in a unidirectional data flow.



Input Layer

Three simulated data files drive the system. `frames.json` contains text descriptions of drone camera observations. `telemetry.json` provides GPS coordinates, altitude, speed, and battery data matched by timestamp. `rules.json` defines the five predefined alert trigger conditions.

Processing Layer

The VLM Processor sends each frame to the Groq API (Qwen3-32B) to identify objects and assess threat level. The Alert Engine checks all five rules against the current frame and history. The Risk Scorer assigns a 0-100 numerical score based on time, location, behavior, and active alerts. The

Pattern Detector performs cross-frame analysis to surface loitering, repeated vehicles, and high-risk zones. The Context Manager maintains memory across all frames so the agent has full session history.

Storage Layer

SQLite provides structured frame indexing queryable by time, location, objects, and alert level. ChromaDB with sentence-transformers (all-MiniLM-L6-v2) provides semantic vector search for natural language queries. The JSON event log maintains a flat file history of all processed events and alerts.

Output Layer

Console logs provide real-time processing output during agent execution. The Streamlit dashboard visualizes all frames, alerts, and the daily report. The Q&A interface allows natural language queries answered by the agent using ChromaDB semantic search combined with LLM reasoning.

2. System Workflow

The flowchart below illustrates the step-by-step processing loop for each surveillance frame — from loading input data through AI analysis, storage, and final report generation.



Step-by-step Description

- 1. Load Input Data** frames.json and telemetry.json are loaded and a timestamp-based map is created to match each frame with its corresponding telemetry entry.
- 2. Next Frame** The agent picks the next unprocessed frame from the list and begins the processing loop.
- 3. VLM Processor** The frame description and telemetry are sent to the Groq API. The model returns detected objects, a one-sentence activity summary, and a threat level (NONE / LOW / MEDIUM / HIGH / CRITICAL).
- 4. Alert Engine** All five predefined rules are checked against the current frame and the session history. Any matching rules generate structured alert objects with severity and message.
- 5. Risk Scorer** A numerical score from 0 to 100 is calculated based on time of day, location type, behavior keywords in the description, and the severity of active alerts.
- 6. Pattern Detector** The context history is scanned for cross-frame patterns such as repeated vehicles, repeated person detections at the same location, and high-frequency alert zones.
- 7. Store Frame** The frame is inserted into SQLite with all metadata, added to ChromaDB as a vector embedding, and logged to the JSON event file.
- 8. More Frames?** If frames remain, the loop returns to Step 2. If all frames are processed, execution continues to report generation.
- 9. Generate Daily Report** The Groq API generates a comprehensive report including statistical summary, natural language surveillance summary, key findings, and security recommendations.
- 10. Q&A Mode** The agent enters an interactive Q&A mode where natural language questions are answered using ChromaDB semantic search and LLM reasoning.